

Arrays in Java (Complete Guide)

1D & 2D Arrays, Memory Layout, Traversal, and Key Algorithmic Solutions

Till now, you have learned variables to store single values, loops to repeat logic, and conditionals to make decisions. Now comes a fundamental Data Structures concept: **Arrays — storing and processing multiple values efficiently in contiguous memory.**

1. What is an Array & Why Does It Matter?

An **array** is a collection of elements of the same data type stored in contiguous memory locations.

```
int[] scores = {85, 90, 78, 92, 88};
```

Memory Representation:

Index	0	1	2	3	4
Value	85	90	78	92	88

Why Arrays Matter:

- Fundamental building block for Data Structures & Algorithms (DSA).
- Used extensively in searching, sorting, and windowing algorithms.
- Ideal for tracking platform analytics, test cases, and grid structures.

2. 1D Array Basics & Common Problems

Declaration, Initialization & Traversal

```
// Declaration & Allocation
int[] arr = new int[5];
int[] arrDirect = {1, 2, 3, 4, 5};

// Traversal Loop
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

Problem 1: Sum & Maximum Elements

```
// Sum of Elements
int sum = 0;
for (int i = 0; i < arr.length; i++) {
    sum += arr[i];
}

// Maximum Element
int max = arr[0];
for (int i = 1; i < arr.length; i++) {
    if (arr[i] > max) {
        max = arr[i];
    }
}
```

Problem 2: Array Reversal (Two-Pointer Technique) & Linear Search

```
// Two-Pointer Reversal
int start = 0, end = arr.length - 1;
while (start < end) {
    int temp = arr[start];
    arr[start] = arr[end];
    arr[end] = temp;
    start++;
    end--;
}

// Linear Search
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == target) {
        return true;
    }
}
```

3. What is a 2D Array?

A **2D array** is an array of arrays structured as a table or matrix containing rows and columns.

Declaration & Accessing:

```
int[][] matrix = new int[3][3];
int[][] matrixDirect = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Accessing element at row 1, column 2 (0-indexed -> value 6):
System.out.println(matrixDirect[1][2]);
```

4. 2D Array Core Problems

Problem 1: Matrix Printing

```
public class Main {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6}
        };

        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Output:

```
1 2 3
4 5 6
```

Problem 2 & 3: Row-wise & Column-wise Sums

```
// Row-wise Sum
for (int i = 0; i < matrix.length; i++) {
    int sum = 0;
    for (int j = 0; j < matrix[i].length; j++) {
        sum += matrix[i][j];
    }
    System.out.println("Row " + i + " sum = " + sum);
}

// Column-wise Sum (Outer loop -> Columns, Inner loop -> Rows)
for (int j = 0; j < matrix[0].length; j++) {
    int sum = 0;
    for (int i = 0; i < matrix.length; i++) {
        sum += matrix[i][j];
    }
    System.out.println("Column " + j + " sum = " + sum);
}
```

Problem 4 & 5: Maximum Element & Matrix Transpose

```
// Maximum Element in 2D Matrix
int max = matrix[0][0];
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        if (matrix[i][j] > max) {
            max = matrix[i][j];
        }
    }
}

// Transpose Matrix (Swapping Rows & Columns)
for (int j = 0; j < matrix[0].length; j++) {
    for (int i = 0; i < matrix.length; i++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

5. 1D vs 2D Comparison & Common Pitfalls

Feature	1D Array	2D Array
Structure	Linear (Single row sequence)	Grid/Table (Rows & Columns)
Access Syntax	<code>arr[i]</code>	<code>matrix[i][j]</code>
Traversal Mechanism	Single <code>for</code> loop	Nested <code>for</code> loops

 **Common Array Mistakes in Interviews**

- **OutOfBounds Exception:** Accessing `arr[arr.length]` instead of `arr[arr.length - 1]`.
- **Boundary Loop Errors:** Using `i <= arr.length` instead of `i < arr.length`.
- **Missing Nested Loop Control:** Forgetting the inner column loop when processing 2D matrices.

 **Practice & Assignment Challenges**

1. **Print Main Diagonal Elements:** Extract values where `i == j` from an \$N\$ imes \$N\$ matrix.
2. **Sum of Diagonal:** Compute total sum of primary and secondary diagonal elements.
3. **Matrix Search:** Check whether a given target integer exists inside a 2D matrix.
4. **Symmetric Matrix Check:** Verify if `matrix[i][j] == matrix[j][i]` for all elements.
5. **Spiral Matrix Traversal:** Traverse and print a 2D matrix in spiral order.